



جامعة أم القرى
UMM AL-QURA UNIVERSITY

Compiler Construction Scanner

Compiler Construction | Department of Computer Science | Jamoum University College

Mayar Faisal Alharbi	445006957
Huda Ali Majrashi	445004905
Jana Ibrahim Alotaibi	445003368

Dr.Ahmed Subahi

Introduction:

A compiler is a software tool designed to transform programs written in high-level programming languages, such as C or Java, into a format that a processor can understand, such as binary instructions or assembly code. This transformation process goes through several sequential phases, beginning with source code analysis, followed by syntactic and semantic processing, and ending with the generation of executable code. Compilers help ensure the correctness of the program by detecting errors and also enhance execution efficiency across different hardware platforms.

The scanner, also known as the lexical analyzer, represents the initial phase of the compilation process. It processes the code as a stream of characters and segments it into meaningful units called tokens. These tokens include elements such as identifiers, reserved keywords, and arithmetic symbols. The scanner ignores whitespace and comments that do not affect program logic. Its design relies on concepts such as regular expressions and deterministic finite automata (DFA) to accurately recognize and classify patterns.

Programming Language:

The language defined in this project is a simplified programming language that consists of:

- **Keywords:** Reserved words like if, int, float, for, else, enum, do, double, default, case.
- **Operators:** Symbols used for operations such as +, -, *, /, ==, !=, >=, <=, !, &, &&, %, |, ||, =, <, >, (,), {, }.
- **Identifiers:** Names that start with an underscore _, followed optional by one or more letters or digits.

Definition:

$$\text{Identifier} \rightarrow _ (L \mid D)^*$$
$$L \rightarrow [a \dots z A \dots Z]$$
$$D \rightarrow [0 \dots 9]$$

- **Number:** $0|0.D^+|JD^*|JD^*.D^+$

$$D=[0 \dots 9]$$

$J=[1\dots 9]$

- End of Word: Represented by a space “ ”.
- End of line: Represented by a special character ” ; “.

Scanner Construction:

In order to draw the DFA, we first wrote and defined the RE. We began creating the code after drawing the DFA, defining the states we have for numbers and identifiers with an enum. Next, as we had previously mentioned with regard to the definition of the RE, we initialized each token that we had in the Constructor. Then, using Java API, the `readFile` method reads and stores each line of the given file. Next, we created the `getLaxeme` method, which uses "space" to split the line into words. The input is then checked using five boolean methods to see if it is a keyword, operator, end of line, letter, or digit.

In order to determine whether the token is a valid number or identifier, the method `executeTransition` for the DFA logic then uses a switch to switch between states based on the current input.

In case S0:

If the input was an underscore '_', return state S7.

If it were 0, return state S2.

And if it was a digit (from 1 to 9), return state S1.

Else return state Se.

In case S1:

If the input was a dot, return state S5.

If it were a digit, stay in S1.

Else return state Se.

In case S2:

If the input was a dot, return state S3.

Else return state Se.

In case S3:

If the input was a digit, return state S4.

Else return state Se.

In case S4:

If the input was a digit, stay in S4.

Else return state Se.

In case S5:

If the input was a digit, return state S6.

Else return state Se.

In case S6:

If the input was a digit, stay in S6.

Else return state Se.

In case S7:

If the input was a letter, stay in S7.

If the input was a digit, stay in S7.

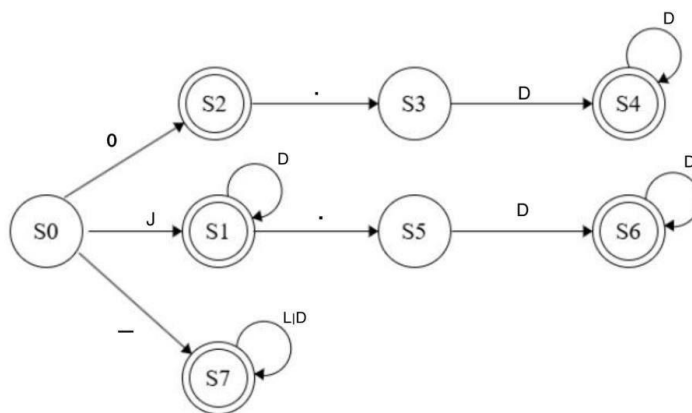
Else return state Se.

Finally, the evaluate method uses if-else to check each word and print the type.

Implementation:

Regular expression:

$0|0.D+|JD^*|JD^*.D+|_(L|D)^*$



```

Appendix1: package newProjectCompiler;

import java.io.IOException;

import java.nio.file.Files;

import java.nio.file.Path;

import java.nio.file.Paths;

import java.util.ArrayList;

enum State {S0, S1, S2, S3, S4, S5,S6,S7,Se}

public class New_project_compiler {

    private ArrayList<String>file_lines;

    private ArrayList<String>keywords;

    private ArrayList<String>operators;

    private String end_of_line;

    private String end_of_word;

    private String fileName;


    public New_project_compiler(String line,String word, String fileName) {

        this.end_of_line = line;

        this.end_of_word = word;

        this.fileName = fileName;

        keywords=new ArrayList<>();

        operators=new ArrayList<>();

        file_lines=new ArrayList<>();

        // Initialize keywords

        keywords.add("case");

        keywords.add("default");

        keywords.add("do"); keywords.add("double");

        keywords.add("else");keywords.add("enum");

        keywords.add("float");keywords.add("for");

```

```

        keywords.add("if"); keywords.add("int");
// Initialize operators
        operators.add("!");operators.add(">");operators.add("<");
        operators.add("=");operators.add("==");operators.add("||");
        operators.add("|");operators.add("+"); operators.add("-");
        operators.add("*");operators.add("/"); operators.add("%");
        operators.add("&&");operators.add("&");operators.add("!=");
        operators.add(">=");operators.add("<=");
        operators.add("{}");operators.add("{}");
        operators.add("(");operators.add(")");

    }

    public void readFile() {
        try {
            Path filePath = Paths.get(fileName);
            Files.lines(filePath).forEach(file_lines::add);
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public String[] getLaxeme (String line,int n){
        String[] a=line.split(" ");
        return a;
    }

```

```

public boolean isKeyword(String lex) {
    return keywords.contains(lex);
}

public boolean isOperator(String lex) {
    return operators.contains(lex);
}

public boolean isEndOfLine(String lex) {
    return end_of_line.equals(lex);
}

public boolean isLetter(char entry) {
    if((entry >= 'A' && entry <= 'Z') || (entry >= 'a' && entry <= 'z'))
        return true ;
    else
        return false;
}

public boolean isDigit(char entry) {
    return entry >= '0' && entry <= '9';
}

public State executeTransition(State currentState, char entry) {
    switch (currentState) {
        case S0:
            if (entry=='_') return State.S7;
            else if (entry=='0') return State.S2;
            else if(entry>='1'&& entry <='9') return State.S1;
            else return State.Se;
    }
}

```

```
    case S1:
        if (entry=='.') return State.S5;
        else if (isDigit(entry)) return State.S1;
        else return State.Se;
    case S2:
        if (entry=='.') return State.S3;
        else return State.Se;
    case S3:
        if (isDigit(entry)) return State.S4;
        else return State.Se;
    case S4:
        if (isDigit(entry)) return State.S4;
        else return State.Se;
    case S5:
        if (isDigit(entry)) return State.S6;
        else return State.Se;
    case S6:
        if (isDigit(entry)) return State.S6;
        else return State.Se;
    case S7:
        if (isLetter(entry)) return State.S7;
        else if (isDigit(entry)) return State.S7;
        else return State.Se;
    default:
        return State.Se;
}
}

public void evaluate(String str,int n) {
```

```

if (isKeyword(str))
    System.out.println(" <" + str + "," + "Keyword" + ">");
else if (isOperator(str))
    System.out.println(" <" + str + "," + "Operator" + ">");
else if (str.equals(end_of_line))
    System.out.println(" <" + str + "," + "End_of_line" + ">");
else {
    State state = State.S0;
    for (char c : str.toCharArray()) {
        state = executeTransition(state, c);
    }
    if (state == State.Se)
        System.out.println(" ERROR..." + str + " Invalid at line : "+n);
    else if (state == State.S7 )
        System.out.println(" <" + str + "," + "Identifier" + ">");
    else if (state == State.S1 || state == State.S4
        || state == State.S6 || state == State.S2)
        System.out.println(" <" + str + "," + "Number" + ">");
    }
}

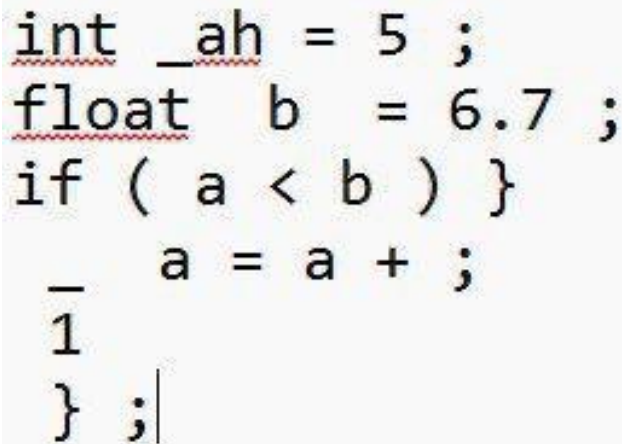
```

```

public static void main(String[] args) {
    String [] words;
    New_project_compiler c= new New_project_compiler(";", " ", "h.txt");
    c.readFile();//1-read source code
    for(int i=0;i<c.file_lines.size();i++){
        words=c.getLaxeme(c.file_lines.get(i),(i+1));
        for(String W:words)

```

```
c.evaluate(W,(i+1));}  
}}
```



```
int _ah = 5 ;  
float b = 6.7 ;  
if ( a < b ) {  
    a = a + 1 ;  
}
```

1. Practical Examples:

The image contains real examples of code written in the programming language for which the compiler was developed, illustrating how the scanner analyzes this code.

2. Symbol Clarification:

The code showcases various symbols such as identifiers, numbers, and operators, providing an opportunity to explain how the scanner identifies each type of element.

3. Error Analysis:

You can use the code to demonstrate how errors are handled, such as incorrect identifier usage or syntax errors. This allows you to indicate how the compiler responds to such mistakes.

4. Connection with DFA:

You can connect the code with the first image (DFA) by explaining how each part of the code corresponds to a specific state in the diagram, enhancing the reader's understanding of the analysis process.

Points to Highlight:

- Identifiers:

How identifiers like `_ah` and `b` are defined.

- Operations:

How operations such as `<` and `=` are used, and how the scanner interacts with these symbols.

- **Code Formatting:**

The importance of adhering to a specific format, highlighting the role of the scanner in accurately analyzing the code.

THE OUTPUT:

Problems @ Javadoc Declaration Console X
<terminated> New_project_compiler (2) [Java Application] C:\Users\HP\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot
<int,Keyword>
<_ah,Identifier>
<=,Operator>
<5,Number>
<;,End_of_line>
<float,Keyword>
ERROR...b Invalid at line : 2
<=,Operator>
<6.7,Number>
<;,End_of_line>
<if,Keyword>
<(,Operator>
ERROR...a Invalid at line : 3
<<,Operator>
ERROR...b Invalid at line : 3
<),Operator>
<},Operator>
<_,Identifier>
ERROR...a Invalid at line : 4
<=,Operator>
ERROR...a Invalid at line : 4
<+,Operator>
<;,End_of_line>
<1,Number>
<},Operator>
<;,End_of_line>

Appendix2: Project Plan

Team Members and Roles:

Huda & Jana:

- Designed regular expressions.
- Wrote token descriptions and categorized them based on functionality.
- Implemented the scanner code and tested token recognition against sample inputs

Mayar :

- Designed the DFA diagram
- Wrote the final project report.

Task	Week
Define the language (Keywords, Operators, Identifiers, Numbers), specify Regular Expressions (RE), and design the DFA and transition states	1
Implement the code in Java, test using input files, and debug the program	2
Prepare the final report and required documentation	3

References:

Cooper, K. D., & Torczon, L. (2012).
Engineering a Compiler. Morgan Kaufmann.

